

Sentencias para la definición de tablas

Módulo: Fundamentos de bases de datos relacionales

|AE4: Implementar estructuras de datos relacionales utilizando lenguaje de definición de datos DDL a partir de un modelo de datos para la creación y mantención de las definiciones de los objetos de una base de datos.

Introducción

En el desarrollo y administración de bases de datos, definir correctamente las tablas es un paso fundamental para garantizar la organización y eficiencia en el almacenamiento de información. Las tablas permiten estructurar datos que a su vez ayudan a establecer las relaciones y dependencias necesarias para mantener la integridad y coherencia de la información. Un diseño sólido facilita el acceso rápido y preciso a los datos, optimizando el rendimiento de las aplicaciones que dependen de ellos.

Comprender cómo crear, modificar y gestionar tablas permite adaptar las bases de datos a las necesidades cambiantes de los proyectos digitales. Al dominar estas herramientas, se logra construir sistemas flexibles y escalables, esenciales en el mundo dinámico de la tecnología.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Conocer el Lenguaje de Definición de Datos DDL
- Definir los Tipos de datos
- Definir e implementar las llaves primarias y foráneas
- Entender el concepto de restricción de nulidad
- Modificar una tabla utilizando DDL
- Modificar un campo en una tabla utilizando DDL
- Eliminar una tabla utilizando DDL
- Truncar una tabla utilizando DDL

Sentencias para la definición de tablas

En el ámbito de la gestión de bases de datos, varios conceptos claves contribuyen a la estructura general y la integridad de los datos: **El Lenguaje de Definición de Datos (DDL)**, los tipos de datos y las **restricciones de nulidad** son algunos de ellos.

DDLs, se refiere a un conjunto de comandos utilizados para definir y manipular la estructura y las características de un esquema de base de datos. Permite a los usuarios crear, modificar y eliminar objetos de base de datos como tablas, índices y vistas. Las sentencias DDL proporcionan la base para organizar los datos y especificar cómo deben almacenarse y acceder a ellos dentro de la base de datos.

Los tipos de datos (números enteros, fechas, booleanos, etc) , por su parte, definen la naturaleza de los datos almacenados en columnas o campos de la base de datos. Seleccionar y asignar correctamente los tipos de datos adecuados es crucial para mantener la integridad de los datos y optimizar el rendimiento de la base de datos.

Además, las restricciones de nulidad rigen si una columna permite o no la presencia de valores nulos. (NULL, NOT NULL).

En general, la combinación de DDL, tipos de datos y restricciones de nulidad **constituye la columna vertebral de la estructura y la gestión de datos de las bases de datos**. DDL proporciona los medios para definir y modificar la estructura de la base de datos mediante sentencias como CREATE TABLE, ALTER TABLE y DROP TABLE.

Data Definition Language (DDL)

El lenguaje de definición de datos (DDL) es un conjunto de comandos o sentencias que se utilizan en los sistemas de gestión de bases de datos (SGBD) para definir y manipular la estructura y las características del esquema de la base de datos. Forma parte del lenguaje SQL (Structured Query Language) utilizado para crear, modificar y eliminar objetos de base de datos como tablas, índices, vistas, restricciones y procedimientos.

El objetivo principal del DDL es definir la estructura lógica y física de la base de datos y sus objetos. Permite a los usuarios o administradores crear, modificar y gestionar el esquema de la base de datos de acuerdo con los requisitos de la aplicación.

Algunas sentencias DDL comunes incluyen:

- **CREATE**: se utiliza para crear nuevos objetos de base de datos como tablas, vistas, índices, procedimientos y funciones.
- **ALTER**: se utiliza para modificar la estructura de los objetos de base de datos existentes. Puede utilizarse para añadir, modificar o eliminar columnas, restricciones, índices u otras propiedades de los objetos.
- **DROP**: se utiliza para borrar o eliminar de la base de datos objetos como tablas, vistas, índices o restricciones.
- **TRUNCATE**: se utiliza para eliminar todos los datos de una tabla, manteniendo intacta su estructura.
- **RENAME**: se utiliza para cambiar el nombre de un objeto de la base de datos.
- **COMMENT**: se utiliza para añadir comentarios o descripciones a los objetos de la base de datos.

Las sentencias DDL suelen ser ejecutadas por administradores de bases de datos o usuarios con los privilegios adecuados. Proporcionan los medios para definir y gestionar la estructura de la base de datos, permitiendo la creación, modificación y eliminación de objetos esenciales para organizar y almacenar datos de forma eficaz.

Crear una tabla y definir campos utilizando (DDL)

Para crear una tabla o definir campos utilizando el Lenguaje de Definición de Datos (DDL) en SQL, puede utilizar la sentencia **CREATE TABLE**. A continuación se muestra un ejemplo de cómo crear una tabla con varios campos:

```
CREATE TABLE TableName
(  Field1 datatype,
  Field2
```

```
datatype,  
Field3  
datatype,
```

Analicemos la sentencia anterior:

- **CREATE TABLE:** Es el comando DDL utilizado para crear una tabla.
- **TableName:** Es el nombre que se da a la tabla.
- **Field1, Field2, Field3, etc:** Son los nombres de los campos o columnas de la tabla.
- **datatype:** Representa el tipo de datos de cada campo. Es necesario especificar un tipo de datos apropiado para cada campo, como VARCHAR para texto, INT para enteros, DATE para fechas, etc.

Ahora vemos un ejemplo más concreto de creación de una tabla llamada "Empleados" con tres campos:

```
CREATE TABLE  
  Employees (  
    EmployeeID INT,  
    FirstName  
    VARCHAR(50),  
    LastName VARCHAR(50)  
  );
```

En este ejemplo, creamos una tabla llamada "Employees" con tres campos: "EmployeeID" de tipo INT, "FirstName" de tipo VARCHAR(50), y "LastName" de tipo VARCHAR(50).

Se pueden añadir más campos a la tabla agregando líneas adicionales en la sentencia CREATE TABLE, siguiendo el mismo patrón.

Hay que tener en cuenta que los tipos de datos específicos y la sintaxis pueden variar ligeramente en función del sistema de gestión de bases de datos que utilice, pero el concepto general de creación de una tabla mediante DDL sigue siendo el mismo.

Tipos de datos

En una base de datos, los tipos de datos definen el tipo de datos que pueden almacenarse en una columna o campo. Especifican la naturaleza de los datos, por ejemplo si son numéricos, textuales o temporales, y determinan el rango de valores que pueden almacenarse. Los distintos sistemas de gestión de bases de datos (SGBD) pueden admitir diferentes tipos de datos, pero a continuación se indican algunos de los más utilizados:

- **Integer (INT)**: Se utiliza para almacenar números enteros (por ejemplo, 1, 10, -5)
- **Decimal/numeric**: Se utiliza para almacenar números con decimales y precisión variable (por ejemplo, 3,14, 10,25).
- **Character/varchar**: Se utiliza para almacenar datos textuales con una longitud fija o variable (por ejemplo, "Juan", "Hola").
- **Boolean**: Se utiliza para almacenar valores verdaderos/falsos o lógicos (por ejemplo, verdadero, falso).
- **Date**: Sirve para almacenar fechas (por ejemplo, "2023-07-13").
- **Time**: sirve para almacenar valores de hora (por ejemplo, "14:30:00").
- **Timestamp/datatime**: Se utiliza para almacenar valores de fecha y hora (por ejemplo, '2023-07-13 14:30:00').
- **Binary**: Se utiliza para almacenar datos binarios, como imágenes o archivos.
- **Enumerated (ENUM)**: Se utiliza para definir una lista de valores permitidos para una columna (por ejemplo, 'Rojo', 'Verde', 'Azul').
- **Object/JSON**: se utiliza para almacenar datos estructurados en formato JSON.
- **Array**: Se utiliza para almacenar múltiples valores en una sola columna (disponible en algunos sistemas de bases de datos).

Estos son sólo algunos de los tipos de datos más utilizados. Dependiendo del DBMS que esté utilizando, puede haber tipos de datos adicionales o variaciones en la sintaxis y el comportamiento. Es importante elegir el tipo de datos adecuado para cada columna para garantizar la integridad de los datos y su almacenamiento eficaz en la base de datos.

Definir una clave primaria ↓

En SQL, las claves primarias son **columnas o combinaciones de columnas** que identifican de forma única cada registro en una tabla.

Para definir una clave primaria en una tabla SQL, puede utilizar la restricción **PRIMARY KEY** en la sentencia **CREATE TABLE** o modificar una tabla existente mediante la sentencia **ALTER TABLE**. Estos son los dos enfoques:

Veamos un ejemplo de definición de la clave primaria durante la creación de la tabla:

```
CREATE TABLE TableName
( Column1 INT
  PRIMARY KEY,
  Column2 VARCHAR,
  ...
);
```

En este ejemplo, la **Column1** se especifica como clave primaria de la tabla **TableName**. Los valores **INT** y **VARCHAR** corresponden al tipo de datos.

Ahora veamos un ejemplo de cómo añadir una clave primaria a una tabla existente:

```
ALTER TABLE TableName
ADD PRIMARY KEY
(Column1);
```

En este ejemplo, **TableName** es el nombre de la tabla existente, y **Column1** es la columna que desea definir como clave primaria. Se pueden añadir varias columnas separándolas con comas dentro del paréntesis.

Es importante saber que una clave primaria **impone la unicidad y garantiza que cada fila de la tabla tenga un valor único** para la columna de clave primaria. Además, las claves primarias tienen un **índice implícito** creado sobre ellas, lo que puede mejorar el rendimiento de la búsqueda.

Al definir una clave primaria, es importante elegir una columna o un conjunto de columnas que identifiquen de forma exclusiva cada fila.

Definir una clave foránea/externa

Una clave externa es una columna o una combinación de columnas de una tabla que establece un vínculo o relación entre dos tablas. Representa una referencia a la clave primaria de otra tabla, creando una relación entre las dos tablas.

Para definir una clave externa en una tabla SQL, puede utilizar la restricción **FOREIGN KEY** en la sentencia **CREATE TABLE** o modificar una tabla existente mediante la sentencia **ALTER TABLE**. Estos son los dos enfoques:

Veamos un ejemplo de definición de la clave Foránea durante la creación de la tabla:

```
CREATE TABLE TableName1 (  
    Column1 INT PRIMARY KEY,  
    ...  
);  
  
CREATE TABLE TableName2 (  
    Column2 Date,  
    ForeignKeyColumn INT,  
    ... FOREIGN KEY (ForeignKeyColumn) REFERENCES  
    TableName1 (Column1)  
);
```

En este ejemplo, **TableName1** es la tabla referenciada que contiene la columna de clave primaria (**Column1**), y **TableName2** es la tabla que contiene la columna de clave externa (**ForeignKeyColumn**).

La restricción **FOREIGN KEY** se define después de la definición de columna de **ForeignKeyColumn** y especifica que hace referencia a **Column1** en **TableName1**.

Los valores **INT** y **DATE** corresponden al tipo de datos.

Ahora veamos cómo añadir una clave foránea a una Tabla existente:

```
ALTER TABLE TableName2  
ADD COLUMN ForeignKeyColumn INT,  
ADD CONSTRAINT fk_foreignkeycolumn  
FOREIGN KEY (ForeignKeyColumn)  
REFERENCES TableName1 (Column1);
```


En este ejemplo:

- 👉 `TableName2` es el nombre de la tabla existente a la que se desea añadir la clave foránea.
- 👉 `ForeignKeyColumn` es la columna en `TableName2` que se definirá como clave externa.
- 👉 La restricción **FOREIGN KEY** se añade a `ForeignKeyColumn` y especifica que hace referencia a `Column1` en `TableName1`, donde `Column1` es la clave primaria.
- 👉 Debes reemplazar `TableName1`, `TableName2`, `ForeignKeyColumn`, y `Column1` por los nombres y columnas adecuados en tu base de datos.

Al definir una clave externa, hay que asegurarse de que la columna a la que se hace referencia en la tabla ya tiene definida una restricción de clave primaria. La restricción de clave externa garantiza la integridad referencial, lo que significa que los valores de la columna de clave externa deben coincidir con los valores existentes en la columna de clave única o clave primaria de la tabla de referencia.

Restricción de nulidad ❌

En SQL, la restricción de nulidad se refiere a la restricción impuesta a una columna para especificar si permite o no valores nulos. Null representa la ausencia de un valor o un valor desconocido en una columna.

Existen dos restricciones de nulidad principales que pueden aplicarse a una columna:

- ➡ **NOT NULL:** Cuando una columna tiene la restricción NOT NULL, significa que la columna debe tener un valor para cada fila, y no se permiten valores nulos. Esta restricción garantiza que la columna siempre contenga datos significativos e impide la inserción de valores nulos.

Ejemplo:

```
Copiar código
CREAR TABLA NombreTabla (
  NombreColumna tipoDato NOT
  NULL,
  ...
);
```

- ➡ **NULL:** Por defecto, si una columna no tiene ninguna restricción de nulidad especificada, permite valores nulos. Esto significa que la columna puede tener tanto valores no nulos como valores nulos.

Ejemplo:

```
Copiar código
CREAR TABLA NombreTabla (
    TipoColumna,
    ...
);
```

Las restricciones de nulidad desempeñan un papel crucial a la hora de garantizar la integridad de los datos y aplicar las reglas de negocio. Al definir explícitamente si una columna permite o no valores nulos, puede establecer reglas para la presencia o ausencia de datos en las tablas de su base de datos.

Modificar una tabla utilizando

DDL ↓

El lenguaje de definición de datos (DDL), incluidas las sentencias ALTER, no se utiliza para modificar datos directamente. Las sentencias ALTER se utilizan para **modificar la estructura de los objetos** de la base de datos, como **tablas, índices o vistas**.

Para modificar datos dentro de las tablas, es necesario utilizar sentencias del Lenguaje de Manipulación de Datos (DML), como INSERT, UPDATE, DELETE y SELECT, como hemos visto en la Lección número 3.

Para que lo entiendas mejor, a continuación te explicamos cómo se utilizan las sentencias ALTER en DDL para modificar la estructura de los objetos de la base de datos:

- ➡ **ALTER TABLE:** Esta sentencia se utiliza para modificar la estructura de una tabla existente, como añadir o eliminar columnas, modificar los tipos de datos de las columnas o alterar las restricciones.

```
ALTER TABLE TableName
ADD ColumnName
DataType;

ALTER TABLE TableName
ALTER COLUMN ColumnName DataType;

ALTER TABLE TableName
DROP COLUMN ColumnName;
```

En este ejemplo:

👉 **"TableName"** es el nombre de la tabla en la que deseas modificar el campo.

👉 **"ColumnName"** es el nombre del campo que deseas modificar

👉 **"NewDataType"** es el nuevo tipo de datos que deseas asignar al campo.

Puedes modificar diferentes aspectos del campo utilizando ALTER TABLE. Algunos ejemplos comunes son:

- Modificar el tipo de datos del campo:

```
ALTER TABLE TableName
ALTER COLUMN ColumnName NewDataType;
```

Modificar una condición de nulidad utilizando DDL

Modificar la restricción de nulidad del campo, para permitir o no permitir valores nulos:

```
ALTER TABLE TableName
ALTER COLUMN ColumnName SET NOT NULL; -- Para no permitir
valores nulos
ALTER TABLE TableName
ALTER COLUMN ColumnName DROP NOT NULL; -- Para permitir
valores nulos
```

Recordá reemplazar "TableName" por el nombre real de tu tabla, "ColumnName" por el nombre del campo que deseas modificar y "NewDataType" por el nuevo tipo de datos que deseas asignar.

Es importante tener en cuenta que las capacidades específicas de modificación del campo pueden variar según el sistema de gestión de bases de datos (SGBD) que estés utilizando. Además, ten en cuenta que la modificación del campo puede tener implicaciones en los datos existentes, como la pérdida de información si se reduce la longitud del campo o la conversión de datos si se cambia el tipo de datos. Asegúrate de realizar copias de seguridad y evaluar cuidadosamente los impactos antes de modificar campos en una tabla.

Es importante volver a destacar que aunque las sentencias ALTER pueden modificar la estructura de los objetos de la base de datos, no modifican directamente los datos de las tablas. Para modificar los datos, deberá utilizar las sentencias DML adecuadas, como INSERT, UPDATE, DELETE o SELECT.

Eliminar una tabla utilizando una DDL

Para eliminar una tabla mediante el lenguaje de definición de datos (DDL), se puede utilizar la sentencia DROP TABLE.

Esta es la sintaxis para eliminar una tabla:

```
DROP TABLE TableName;
```

En esta sentencia, "TableName" es el nombre de la tabla que desea eliminar. Asegurate de sustituir "TableName" por el nombre real de la tabla que deseas eliminar.

Hay que tener en cuenta los siguientes puntos cuando se utiliza la sentencia **DROP TABLE**:

- Al eliminar una tabla se borrarán permanentemente todos los datos y la estructura de la tabla. Por lo tanto, tenga cuidado al utilizar esta sentencia, ya que los datos no se pueden recuperar una vez eliminada la tabla.
- Algunos sistemas de gestión de bases de datos (SGBD) pueden requerir permisos o privilegios específicos para ejecutar la sentencia DROP TABLE.
- Si la tabla tiene objetos dependientes, como vistas, índices o restricciones, también se eliminarán cuando se elimine la tabla.

A continuación se muestra un ejemplo de cómo eliminar una tabla denominada "Employees":

```
DROP TABLE Employees;
```

La ejecución de esta sentencia eliminará la tabla "Employees" y todos sus datos asociados.

Es de suma importancia tener cuidado al utilizar la sentencia DROP TABLE, ya que elimina irreversiblemente la tabla y sus datos de la base de datos. Asegúrate siempre de tener una copia de seguridad de los datos y comprobar que estás eliminando la tabla correcta antes de ejecutar la sentencia.

Truncar una tabla utilizando DDL

Para truncar una tabla utilizando el Lenguaje de Definición de Datos (DDL), podés utilizar la sentencia TRUNCATE TABLE.

Esta es la sintaxis para truncar una tabla:

```
TRUNCATE TABLE TableName;
```

En esta sentencia, "TableName" es el nombre de la tabla que desea truncar. Asegúrate de sustituir "TableName" por el nombre real de la tabla que deseas truncar.

Al truncar una tabla se eliminan todas las filas y datos de la misma, manteniendo intacta la estructura de la tabla. Es una operación más rápida que eliminar todas las filas una a una mediante la sentencia DELETE.

Tenga en cuenta los siguientes puntos cuando utilice la sentencia TRUNCATE TABLE:

- Truncar una tabla es una operación irreversible. Una vez truncados los datos, no se pueden recuperar. Es importante realizar copias de seguridad o asegurarse de tener una copia de los datos antes de truncar una tabla.
- Truncar una tabla suele requerir permisos o privilegios adecuados, dependiendo del sistema de gestión de bases de datos (SGBD) que esté utilizando.

A continuación veamos un ejemplo de cómo truncar una tabla denominada "Employees":

```
TRUNCATE TABLE Employees;
```

Ejecutando esta sentencia se eliminarán todas las filas y datos de la tabla "Employees" conservando su estructura.

Cierre

Durante esta lección hemos hecho un recorrido por las sentencias para la definición de tablas. Aprendimos sobre la sintaxis básica para la creación de expresiones DDL que resuelven un requerimiento de definición de datos, hemos visto cómo construir sentencias de creación de tablas utilizando DDL y definir campos, tipos de datos, nulidad, llaves primarias y foráneas de acuerdo a un modelo de datos existente para satisfacer un requerimiento y construir sentencias utilizando DDL para modificar atributos de una tabla.

Referencias

- Database Journal. (n.d.). What is MySQL?
<https://www.databasejournal.com/mysql/what-is-mysql/>
- Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). Database systems: The complete book (2nd ed.). Pearson.
- Kroenke, D. M., & Auer, D. J. (2014). Database concepts (7th ed.). Pearson.
- Microsoft. (n.d.). SQL Server documentation.
<https://docs.microsoft.com/en-us/sql/>
- Mode Analytics. (n.d.). SQL tutorial. <https://mode.com/sql-tutorial/>
- SQLCourse.com. (n.d.). SQL tutorial – Beginner & advancer.
<https://www.sqlcourse.com/>

¡Muchas gracias!

Nos vemos en la próxima lección

